

Large Language Models (LLM)

Core concepts, architecture, training pipeline & interview essentials

1. What is a Large Language Model (LLM)?

- A neural network (typically a Transformer) trained on massive text corpora to predict the next token in a sequence — this simple objective, at scale, produces broad language understanding and generation ability.
- "Large" refers to parameter count (billions to trillions) and training data size (trillions of tokens).
- Examples: GPT-4/4o, Claude, Gemini, Llama 3, Mistral, DeepSeek, Qwen.

2. Core Architecture — The Transformer

- **Self-Attention:** Each token computes attention scores against every other token, letting the model weigh which words matter most for context (captures long-range dependencies better than RNNs).
- **Multi-Head Attention:** Several attention computations run in parallel on different learned projections, capturing different types of relationships simultaneously.
- **Positional Encoding:** Since attention has no inherent sense of order, position info (sinusoidal or rotary/RoPE) is injected so the model knows word order.
- **Feed-Forward Layers:** Position-wise fully connected layers applied after attention in each block, adding non-linear transformation capacity.
- **Decoder-only architecture:** Most modern LLMs (GPT, Llama, Claude) use a decoder-only stack — causal (masked) self-attention so each token only attends to previous tokens, enabling autoregressive generation.
- **Residual connections + LayerNorm:** Stabilize training across dozens of stacked layers.

3. How LLMs Are Trained

- **Pre-training:** Self-supervised next-token prediction on massive unlabeled text/code corpora. This is where most compute cost and general knowledge is acquired.
- **Supervised Fine-Tuning (SFT):** Model is fine-tuned on curated (prompt, response) pairs to follow instructions.
- **Alignment — RLHF / DPO:** Reinforcement Learning from Human Feedback trains a reward model on human preference rankings, then optimizes the LLM (via PPO) against it. Direct Preference Optimization (DPO) is a simpler, more stable alternative that skips the separate reward model.
- **RAG / tool-use tuning:** Many production LLMs are further tuned to call tools/functions and cite retrieved context reliably.

4. Key Concepts & Terminology

Term	Meaning
Tokenization	Splitting text into sub-word units (e.g. Byte-Pair Encoding / BPE) that the model reads/writes; not the same as words.
Embedding	Dense vector representation of a token/text that captures semantic meaning.
Context Window	Max number of tokens (input + output) the model can attend to at once (e.g. 128K, 200K, 1M).
Parameters	Learned weights of the network; rough proxy for model capacity (e.g. 8B, 70B, 400B+).
Temperature	Sampling randomness control — low = deterministic/focused, high = creative/diverse.
Top-k / Top-p	Sampling restricted to the k most likely tokens, or the smallest set whose cumulative probability $\geq p$ (nucleus sampling).
Logits	Raw, un-normalized output scores before softmax converts them to a probability distribution over the vocabulary.
Quantization	Reducing weight precision (FP16 $\rightarrow$ INT8/INT4) to shrink model size and speed up inference with minimal quality loss.
Mixture of Experts (MoE)	Architecture where only a subset of specialized sub-networks ('experts') activate per token, boosting capacity without proportional compute cost.
Distillation	Training a smaller "student" model to mimic a larger "teacher" model's outputs.

5. Prompting Techniques

- **Zero-shot:** Ask directly with no examples.
- **Few-shot:** Provide a handful of input/output examples in the prompt to steer format and style.
- **Chain-of-Thought (CoT):** Ask the model to reason step-by-step before answering — improves performance on multi-step/logic tasks.
- **System prompt:** A persistent instruction that sets role, tone, and constraints for the whole conversation.
- **Self-consistency:** Sample multiple reasoning paths and take the majority answer to reduce error.

6. Popular LLM Families

Family	Maker	Notable Trait
GPT-4o / o-series	OpenAI	Strong general reasoning; o-series adds extended "thinking" for hard problems.
Claude (Sonnet/Opus/Haiku)	Anthropic	Long context, strong coding & agentic tool-use, constitutional-AI alignment.
Gemini	Google DeepMind	Natively multimodal, very large context windows, tight Google ecosystem integration.
Llama 3/4	Meta	Open-weight, widely fine-tuned by the community, strong for self-hosting.
Mistral / Mixtral	Mistral AI	Efficient open-weight models; Mixtral popularized open MoE.
DeepSeek	DeepSeek	Open-weight, strong reasoning/coding at low training cost via MoE + RL.

7. Evaluating LLMs

- **Perplexity:** Measures how well a model predicts held-out text (lower = better); mainly used for pre-training/language-modeling quality, not task ability.
- **Benchmark suites:** MMLU (broad knowledge), HumanEval / MBPP (code generation), GSM8K / MATH (math reasoning), HellaSwag (commonsense), MT-Bench / Chatbot Arena (human/LLM-judged conversation quality).
- **LLM-as-judge:** Using a strong LLM to score/compare outputs at scale — fast but can inherit the judge model's own biases.
- **Production evals:** Task-specific accuracy, hallucination/fairfulness rate, latency, and cost-per-request matter more than public leaderboards for real applications.

8. Limitations & Challenges

- **Hallucination:** Model generates plausible-sounding but false or unverifiable information — it predicts likely text, not verified fact.
- **Knowledge cutoff:** No awareness of events after the training data snapshot, unless given external/retrieved context.
- **Context window limits:** Long documents/conversations may exceed the window or degrade in "needle in a haystack" recall.
- **Bias:** Reflects biases present in training data.
- **Cost & latency:** Larger models are expensive and slower to run at scale; production systems must balance quality vs. cost/latency.
- **Reasoning brittleness:** Can fail on novel arithmetic, planning, or logic tasks that fall outside common training patterns.

9. Quick-Fire Interview Answers

- **Why decoder-only vs. encoder-decoder?** Decoder-only is simpler to scale and works well for open-ended generation; encoder-decoder (e.g. original Transformer, T5) suits well-defined seq-to-seq tasks like translation.
- **Why is attention cost quadratic?** Every token attends to every other token, so cost grows as $O(n^2)$ with sequence length n — a key reason long-context models need optimizations (sparse/linear attention, sliding window, RoPE scaling).
- **Fine-tuning vs. prompting vs. RAG — when to use which?** Prompting for quick behavior changes; RAG when the model needs fresh/private knowledge; fine-tuning when you need consistent style, domain-specific behavior, or to bake in patterns cheaper than long prompts at inference time.
- **What is a hallucination mitigation strategy?** Grounding via RAG, citing sources, lower temperature, self-verification prompts, and structured output validation.
- **What are emergent abilities?** Capabilities (e.g. multi-step reasoning) that appear unpredictably once model/data scale crosses a threshold, rather than improving smoothly — an active research debate on whether this is a real phase-change or a measurement artifact.
- **What is RLHF's main weakness?** The reward model can be "gamed" (reward hacking), and human preference data is expensive and can encode annotator bias — DPO and RLAIF (AI-generated feedback) are common mitigations.
- **One-line summary:** An LLM is a Transformer-based next-token predictor, scaled up and aligned with human feedback, that turns statistical language patterns into general-purpose reasoning and generation ability.